

Session 1: The Processor (CPU) - Architecture and Execution

Chapter 1: The Transistor - The Switch That Changed the World ⚡	2
1. What is a Transistor?	3
2. How do we convert current into Logic?	3
3. Silicon - The "Magic" Material	3
💡 QualiAdept Analogy: "Water Dam"	3
🔍 The Tester Eye: Why does it break?	4
📏 Reflection exercise	4
Chapter 2: Von Neumann Architecture - The Foundation of Every Modern Computer ⌚	4
1. The Revolution: The "Stored-Program Concept"	4
2. The 5 Fundamental Components	5
3. „Gâtuirea" Von Neumann (The Bottleneck)	5
💡 QualiAdept Analogy: "The Carpentry Workshop"	5
🔍 Tester's Eye: Where do system errors occur?	6
📏 Reflection exercise	6
Chapter 3: Internal Units - ALU and CU (The Calculator and the Manager) ⚙️	7
1. ALU (Arithmetic Logic Unit) - "The Working Mathematician"	7
2. CU (Control Unit) - "Orchestra Conductor"	7
3. Internal Clock (The System Clock)	8
💡 QualiAdept Analogy: "Construction Site"	8
🔍 The Tester's Eye: Where do calculation errors occur?	8
📏 Reflection exercise	9
Chapter 4: Registers - Ultra-fast memory next to the "heart" ⚡	10
1. What is a Registry?	10

2. The 5 “Star” Registers (The Processor’s Bodyguard)	10
3. Speed Hierarchy: Why don't we do everything from Registers?	11
💡 QualiAdept Analogy: “The Chef and the Pockets”	11
🧑‍🔬 The Tester's Eye: "Overflow" and Data Corruption	11
📝 Reflection exercise	12
Chapter 5: The Instruction Cycle (Fetch-Decode-Execute) 🔄	13
1. The three fundamental stages (Plus one).....	13
A. Fetch.....	13
B. Decode.....	13
C. Execute	14
D. Store (Storage / Writing - Optional).....	14
💡 QualiAdept Analogy: “The Sushi Master and the Recipe”	14
🧑‍🔬 The Tester's Eye: Infinite Loop and "Lag"	14
📝 Reflection exercise.....	15
Chapter 6: Frequency, Cores and Threads 🚀	15
1. Frequency (Clock Speed) - Step speed	16
2. Cores - More Physical Workers	16
3. Threads - Efficiency at the Detail Level.....	16
💡 QualiAdept Analogy: “The Kitchen with Many Chefs”	17
🧑‍🔬 Tester's Eye: Race Conditions and Blockages.....	17
📝 Reflection exercise	17
Chapter 7: Manufacturing Process (Nanometers) and Moore's Law 🔬	18
1. Photolithography: How to "print" a Processor	19
2. What does Nanometers (nm) mean?	19
3. Moore's Law.....	19
💡 QualiAdept Analogy: “Drawing on the Grain of Rice”	20
🧑‍🔬 The Tester's Eye: "Binning" – Not all chips are created equal.....	20
📝 Reflection exercise	20

Chapter 1: The Transistor - The Switch That Changed the World ⚡

💡 If you want to understand how a computer works, you have to forget about screens, mice, and applications. You have to get down to the microscopic level, where it's all about **electricity**. The "magic" of technology begins with a tiny piece called **Transistor**.

1. What is a Transistor?

Imagine a light switch on the wall. It has two states: **On (1)** or **Off (0)**.

The transistor is exactly this, but with three major differences:

- **Size:** It is so small that billions of transistors fit on a surface the size of a fingernail.
- **Speed:** It can close and open billions of times in a single second.
- **Control:** It doesn't require a human finger to press it; it activates using an electrical impulse.

2. How do we convert current into Logic?

A single transistor can't do much. But if we connect two transistors together, we can create a logical rule.

- *Example:* "If transistor A is on **AND** transistor B is on, then it lets the current pass through." This is the zero moment of computer science. From the combination of billions of such "switches", we can represent anything: a letter, a color in a picture, or a sound.

3. Silicon - The "Magic" Material

Transistors are made of **Silicon** (which is found in sand). Silicon is a *semiconductor*.

- **Special property:** In its natural state it does not conduct current, but if we add certain chemical impurities to it, we can make it conduct current only when we want it to. This ability to control electricity is what allows the machine to "think".

💡 QualiAdept Analogy: "Water Dam"

- 💡 Imagine a canal with water flowing through it.
- The channel has a gate (the transistor).
- If the gate is raised, water passes (Signal 1).
- If the gate is lowered, the water stops (Signal 0).

Now imagine a huge system with billions of these interconnected gates. Opening one gate

determines whether water reaches the next gate. Through this game of "water passes/water does not pass," we can create complex code. The computer is nothing more than an incredibly complex hydraulic system, but instead of water, we use **ELECTRON**.

The Tester Eye: Why does it break?

Transistors are so small that they can be affected by temperature or cosmic radiation.

- If a transistor "stucks" in the Off (0) state, the final calculation will be wrong.
- In modern processors, if a transistor a few nanometers thick burns out due to excessive heat, the entire circuit can become unusable. That's why cooling the processor is not just an option, but a vital necessity to maintain correct logic.

Reflection exercise

Think about a light bulb in your house. If you had 8 bulbs and could turn them on and off in various combinations, how many different "messages" could you send to a neighbor across the street?

(The answer is 256 different combinations – this is the basis of a Byte, which we will study later).

Chapter 2: Von Neumann Architecture - The Foundation of Every Modern Computer

 Until the mid-1940s, computers were physically "programmed." If you wanted the machine to do a different calculation, you had to move wires, change switches, and rebuild the circuits. Everything changed thanks to the mathematician **John von Neumann**, who proposed a model so efficient that it is still used today in your smartphone, your laptop, and NASA supercomputers.

1. The Revolution: The “Stored-Program Concept”

Von Neumann's greatest innovation was the idea that **instructions** (what the computer needs to do) and **DATES** (the numbers they work with) can be stored in the same place: **Memory**.

Before him, the program was "hardwired." After him, the computer became a universal machine: you give it a set of instructions for math, it becomes a calculator; you give it instructions for text, it becomes a word processor.

2. The 5 Fundamental Components

The Von Neumann architecture defines a system consisting of 5 parts that communicate with each other:

- **Central Processing Unit (CPU):** The heart of the system (detailed in the following chapters).
- **Memory Unit:** Where instructions and data are stored (RAM).
- **Input Unit:** How we "talk" to the machine (Keyboard, Mouse, Sensors).
- **Output Unit:** How the machine responds to us (Monitor, Printer, Speakers).
- **Magistralele (Buses):** The "cables" or data highways that connect all these pieces.

3. „Gâtuirea” Von Neumann (The Bottleneck)

Although brilliant, this architecture has a famous weakness. Because instructions and data travel on the same "highway" (highway) between memory and the processor, they cannot pass simultaneously.

- **Problem:** The processor is much faster than memory. Often, the "brain" sits idle and waits for data to come from memory.
- **The modern solution:** This is where the memories came from **Cache**, which we will talk about in Session 2, to shorten this waiting time.

💡 QualiAdept Analogy: “The Carpentry Workshop”

💡 Imagine a very well-organized carpentry workshop:

- **The carpenter (CPU):** He is the one who executes the work.
- **Workbench (Registers):** Where does he keep the piece he's polishing right now?
- **Material shelf (RAM memory):** Here you will also find **boards** (Data), but also **instruction manual** (The program).
- **Service door (Input/Output):** This is where the raw wood comes in and this is where the finished furniture comes out.

Von Neumann's magic: Previously, if the carpenter wanted to make a chair instead of a table,

he had to tear down the workshop and rebuild it. In the Von Neumann model, he simply closes the "How to Make a Table" manual and opens the "How to Make a Chair" manual. The workshop remains the same; only the instructions in memory change.

Tester's Eye: Where do system errors occur?

Because data and instructions reside in the same memory, a serious programming error can cause the processor to "confuse" data with instructions.

- **Example:** If a program accidentally writes data over the area where the instructions are, the processor will try to "execute" a phone number or an image as if it were an order. This is when the application suddenly closes (Crash) or the famous "Blue Screen of Death" appears. A good tester checks if the program respects the memory boundaries!

Reflection exercise

Think of an automated teller machine (ATM).

- What does it represent? **Input-ul**?
- What does it represent? **Data** from memory?
- What does it represent? **Instructions** (The program)?
- What does it look like? **Output-ul**?

Notice how the same "box" can make a cash withdrawal, a PIN change, or a balance inquiry, just by changing the instruction manual in its memory.

Chapter 3: Internal Units - ALU and CU (The Calculator and the Manager)

 If we open a modern processor, we will find an extremely dense structure, but it all comes down to two large departments that work in perfect symbiosis: **GO** (the one who performs the hard work) and **WITH** (the one who knows what work needs to be done). Without this division, the processor would be either a genius without direction or a manager without subordinates.

1. ALU (Arithmetic Logic Unit) - "The Working Mathematician"

The ALU is the heart of execution. Everything that means calculation in the digital universe happens here. It knows how to do two types of operations:

- **Arithmetic Operations:** Addition, subtraction, multiplication and division.
 - *The QualiAdept secret:* In reality, ALU only knows how to do **ASSEMBLY** at the bit level. Subtraction is an addition with negative numbers, and multiplication is an addition repeated. Everything is reduced to its simplest form for maximum speed.
- **Logical Operations:** Comparisons like "Is A greater than B?", "Are the two values equal?", or operations like **AND**, **OR**, **NOT**.
 - These are the "decisions" that the computer makes. For example: "IF the entered password IS EQUAL TO the saved password, THEN allow access."

2. CU (Control Unit) - "Orchestra Conductor"

The Control Unit does not do calculations. Its role is to manage data traffic and give orders. It is the "brain of the brain".

- **Interpretation:** The CU receives the instruction from memory (Fetch) and "translates" it (Decode). It understands whether bit 0110 means "add" or "erase".
- **Synchronization:** It sends electrical signals to the ALU, Memory, or Input/Output devices to tell them when to act.
- **Flow management:** Decides what the next instruction to be executed is.

3. Internal Clock (The System Clock)

ALU and CU do not work chaotically. They are synchronized by a **Quartz Oscillator** (system clock).

- Each "tick" of the clock is an opportunity for the CU to send an order and for the ALU to make a calculation.
- When we talk about a 3.6 GHz processor, we are talking about a "conductor" that issues 3.6 billion commands per second.

💡 QualiAdept Analogy: "Construction Site"

💡 Imagine a construction site where a complex house is being built:

- **ALU is the Worker.** He has the tools, he drives the nails, he carries the bricks. He is the only one who produces physical results. If you give him two planks and say, "Join them," he does it.
- **CU is the Chief Engineer.** He stands with the house plan (the Program) in his hand. He does not pick up the trowel, but he tells the Worker: "Now take brick A and put it on top of B." He also tells the crane (Input/Output) when to bring in new materials.

Magic: If the Engineer (CU) misreads the plan, the Worker (ALU) will build a crooked wall without asking questions. The Worker is perfect, but he blindly "listens" to the signals he receives.

👮 The Tester's Eye: Where do calculation errors occur?

In testing, ALU-related errors are rare (the processor rarely gets the math wrong), but ALU-related errors are **WITH** (logic) are common.

- **Race Conditions:** When the CU tries to issue commands faster than the ALU can execute

or when two processes fight over the same resource.

- **Overclocking:** If you force the Clock to tick too fast, the Engineer issues orders so quickly that the Worker (ALU) starts dropping tools or misplacing bricks due to the speed. The result? An unstable system or corrupted data.

Reflection exercise

Imagine you want to calculate the average of two grades (10 and 8).

- What instructions should he give? **WITH?** (Ex: Bring note 1, Bring note 2, Tell ALU to add them, Divide by 2).
- What calculation does it actually do? **GO?**

Do you now understand why, even for a simple average, the "worker" and the "engineer" must collaborate closely?

Chapter 4: Registers - Ultra-fast memory next to the "heart" ⚡

 In previous chapters we saw that data resides in Memory (RAM) and is processed in the ALU. But there is a speed problem: RAM is "far" from the processor. To make an instant calculation, the processor needs a tiny but incredibly fast internal memory. These are **records**.

1. What is a Registry?

A registry is a temporary storage location located **directly inside the processor**.

- **Speed:** It is the fastest form of memory in the entire digital universe. Access to a register occurs in fractions of a nanosecond.
- **Capacity:** It is extremely small. A register usually stores only a single number or instruction (32 or 64 bits).
- **Role:** Remember the data that the ALU processes **in this very microsecond**.

2. The 5 “Star” Registers (The Processor’s Bodyguard)

Although there are many registers, a few are essential to understanding how the "magic" works:

- **PC (Program Counter):** Remember the memory address of **next** instructions. It's like a finger that always points to where I left off in the book.
- **IR (Instruction Register):** Remember the instruction being executed**now**(after being brought from RAM).
- **ACC (Accumulator):** It is the “scoreboard.” This is where the ALU stores the result of each intermediate calculation.

- **MAR (Memory Address Register):** It contains the address in RAM from where we want to read or write data.
- **MDR (Memory Data Register):** It is the "container" that contains the actual data that has just arrived from RAM or that is about to be sent to RAM.

3. Speed Hierarchy: Why don't we do everything from Registers?

If registers are so fast, why do we still need RAM or SSD?

- **Cost:** A single register bit is extremely expensive to manufacture (it takes up a lot of space on the silicon crystal).
- **Space:** If we made a computer out of registers only to have 8GB of storage, the processor would be the size of a building and would consume as much electricity as a city.

QualiAdept Analogy: "The Chef and the Pockets"

Let's go back to our kitchen analogy:

- **Storage (SSD) is Pantry:** Far, far, but you have to walk there (Slowly).
- **RAM is the Workbench:** Closer, I can fit many ingredients on it (Medium speed).
- **The registers are the chef's Hands and Pockets:**
 - If the chef cuts an onion, the onion is in **hands** (Data register).
 - If he needs salt immediately, he takes it out of the **pocket** (Other register).
 - He can't hold 10 kg of potatoes in his pockets, but he can hold exactly the salt he needs for the spoon he's mixing **now**.

Magic: The processor only "thinks" with what it has in its hands (Registers). If it needs something from the counter (RAM), it has to leave what it has in its hands and make a swap.

The Tester's Eye: "Overflow" and Data Corruption

 In testing, one of the most famous errors is **Integer Overflow**.

- **How it appears:** Each register has a fixed size (e.g. 8 bits can hold a maximum of 255).
- **Error:** If you tell the processor to add 1 to the number 255 in an 8-bit register, it will not become 256 (because there is no room), but will "jump" back to 0.
- **Impact:** Imagine a video game where your score is at its maximum and suddenly becomes 0, or a banking system where a huge amount becomes negative due to the limitation of

the registers. A good tester always checks what happens at the boundary values!

Reflection exercise

If your processor is 64-bit, that means its "hands" (registers) can grab a piece of information 64 binary digits long in one go.

- *Why do you think a 64-bit system is faster than a 32-bit one for complex calculations?*

Chapter 5: The Instruction Cycle (Fetch-Decode-Execute)

🧠 So far we have seen the "parts" (ALU, CU, Registers) and the "construction plan" (Von Neumann). Now it's time to see how it all works in motion. **The Instruction Cycle** It's the heartbeat of any computer. Whether you're writing an email, watching a movie, or sending a rocket to the moon, the processor performs the same set of steps over and over again.

1. The three fundamental stages (Plus one)

Every software instruction goes through a mandatory circuit before producing an effect. Although it happens in nanoseconds, the process is extremely rigorous:

A. Fetch

The processor needs to know what to do.

- Control Unit (**WITH**) check **Program Counter (PC)** to find out the memory address of the next instruction.
- The address is sent via the bus to the RAM.
- The RAM sends back the contents of that address, which is stored in **Instruction Register (IR)**.
- The PC increments (increases by 1) to be ready for the next instruction.

B. Decode

The instruction arriving from memory is just a string of bits (ex: 01101010).

- **Control Unit** analyze this code.
- It translates bits into specific electrical signals.
- *Example:* "Code 0110 means: Take the number from Register A and add it to the number from Register B."

C. Execute

This is when the mathematical or logical "magic" occurs.

- The Control Unit sends signals to **GO**.
- The ALU performs the operation (addition, comparison, etc.).
- If additional data is needed from RAM, it is fetched now.

D. Store (Storage / Writing - Optional)

The calculation result is stored for later use.

- The result of **Battery (ACC)** it is sent back to RAM or remains in a register for the next step.

💡 QualiAdept Analogy: "The Sushi Master and the Recipe"

💡 Imagine an ultra-fast sushi master working in a restaurant:

- **Fetch:** The master looks at the command list (RAM) and reads the first command: "Command #45".
- **Decode:** He looks in his recipe book (CU) and realizes that #45 stands for "Salmon Nigiri." He realizes he needs a knife, rice, and fish.
- **Execute:** Cut the fish, form the rice and join them (ALU). This is the actual work.
- **Store:** Place the piece of sushi on the plate, ready to be delivered to the waiter (Output/Memory).

The secret: The master does this so quickly that, to an outside observer, the plates appear on the tape instantly. But he never skips the step of reading the recipe!

🕵️ The Tester's Eye: Infinite Loop and "Lag"

In testing and debugging, understanding this cycle helps us identify serious problems:

- **Infinite Loop:** It occurs when the instruction at step 100 tells the PC to jump back to step 50, and there is no stopping condition at step 50. The processor will execute the same steps infinitely, "freezing" the application.
- **CPU Bottleneck:** If the instructions are very complex (Execute takes a long time) or if

fetching them from RAM (Fetch) is slow, the processor is “idling.” As a tester, you notice this when the processor is at 100% load, but the application interface does not move.

Reflection exercise

Think of a video game. When you press the "Jump" key, the processor has to go through this cycle:

- **Fetch:** Read the code for "Check key".
- **Decode:** Understands that the X key is pressed.
- **Execute:** Calculates the character's new position on the screen.
- **Store:** Save the new position in memory so the graphics card can draw it.

If your processor is 3.0 GHz, how many times can it complete this cycle in one second for your character?

Chapter 6: Frequency, Cores and Threads

🌀 So far we have studied a processor as if it were a single entity doing a single thing. But the modern world demands that the computer listen to music, download a file, run an antivirus and display a game, all at the same time. To do this, engineers have resorted to two strategies: increasing raw speed (Frequency) and multiplying the work arms (Core and Threads).

1. Frequency (Clock Speed) - Step speed

Frequency, measured in **Hertz (Hz)**, represents the number of Fetch-Decode-Execute cycles that a processor can complete in one second.

- **1 Hz** = 1 cycle per second.
- **3.5 GHz** = 3,500,000,000 (3.5 billion) cycles per second.

Physical Limit: Why don't we have 100 GHz processors? Because of heat. The faster the transistors turn on/off, the more heat they produce. If we exceed a certain threshold (about 5 GHz for standard silicon), the processor would melt instantly. That's why the industry has gone from "faster" to "more".

2. Cores - More Physical Workers

And **Core** it is essentially a complete processor (with its own ALU, CU, and registers) integrated on the same silicon chip.

- **Single-core:** A single worker. If he wants to do two tasks, he has to jump from one to the other very quickly (multitasking by switching).
- **Multi-core (Dual, Quad, Octa-core):** Multiple physical workers that can process completely different tasks at the exact same time (True Parallelism).

3. Threads - Efficiency at the Detail Level

And **Thread** (thread) is a virtual unit. Technologies such as *Hyper-Threading* (Intel) or *SMT* (AMD) allow a single physical core to behave as if it were two logical cores.

- **How does it work?** While one thread is waiting for data to come from RAM (the Fetch step), the kernel is not sitting idle; it uses those free nanoseconds to process instructions from the second thread.

- *The result:* A processor with 8 cores and 16 threads can manage the data flow more efficiently, avoiding dead times.

💡 QualiAdept Analogy: “The Kitchen with Many Chefs”

💡 Let's go back to our sushi restaurant to clarify the differences:

- **Frequency:** It's the speed at which a single chef moves their hands. If they move faster, they finish the order faster.
- **Nuclei (Cores):** It represents the number of chefs in the kitchen. If you have 4 chefs (Quad-core), you can prepare 4 plates of sushi simultaneously.
- **Threads (Hyper-threading):** Imagine a chef has two orders in front of him. While the rice for the first order is cooking (waiting for data from RAM), the chef is cutting the fish for the second order. He still has two hands (only one physical core), but he organizes his work so well that it seems like he is working on two plates at once.

Careful! If a recipe can't be shared (for example, you can't have two cooks peel the same carrot at the same time), there's no point in having 100 cooks. Some programs are "Single-threaded" and will run just as slowly on a 64-core processor.

👤 Tester's Eye: Race Conditions and Blockages

In testing, the switch to multi-core brought a new category of bugs: **Concurrency Issues**.

- **Race Condition (Competition):** It occurs when two cores try to modify the same piece of data at the same time.
 - *Example:* Core 1 reads the balance of 100 lei and wants to subtract 10. In the exact same millisecond, Core 2 also reads 100 lei and wants to add 50. If they are not synchronized, the final result could be 90 or 150, instead of the correct 140.
- **Deadlock:** Thread A waits for Thread B, and Thread B waits for Thread A. The system "freezes" completely, even though the processor is not at 100% load.

🔧 Reflection exercise

Open "Task Manager" (Windows - Ctrl+Shift+Esc) or "Activity Monitor" (Mac). Go to the Performance/CPU tab.

- How many physical cores do you see?
- How many Logical Processors/Threads do you see?
- Notice the graph: are all cores used equally while just browsing the internet?

You will notice that, most of the time, some cores "sleep" while only one does the heavy lifting for the operating system.

Chapter 7: Manufacturing Process (Nanometers) and Moore's Law

 If we scaled up a modern processor to the size of a city, we would see streets and structures more complex than anything humans have ever built. But this “city” is created on a piece of silicon the size of a fingernail. How do we manage to “draw” billions of transistors in such a small space? And how much smaller can we get before the laws of physics stop us?

1. Photolithography: How to "print" a Processor

Processors are not assembled by robots that put small parts together; they are "printed" using light.

- **The process:** A pure silicon disk (called *wafer*). This is coated with a photosensitive substance.
- **Lumina UV:** A highly precise ultraviolet light is projected through a mask (a kind of photographic negative) onto the disk. The light "burns" the circuit pattern onto the silicon.
- **Engraving:** The exposed areas are then chemically treated to create the channels through which the electrons will flow.

2. What does Nanometers (nm) mean?

When you hear about "7nm process" or "5nm," the number refers to the approximate size of the transistor components (specifically the gate through which current flows).

- **Comparative size:** A nanometer is a billionth of a meter.
 - A human hair is ~80,000 nm wide.
 - Un virus are ~100 nm.
 - A modern transistor is ~5 nm (i.e. it is the size of a few dozen silicon atoms placed side by side).

Why is smaller better?

1. **Density:** More transistors in the same space = more computing power.
2. **Efficiency:** Electrons have a shorter distance to travel = higher speed and lower energy consumption.

3. Moore's Law

In 1965, Gordon Moore (co-founder of Intel) noticed that the number of transistors on a chip was doubling approximately every two years, while the cost was decreasing.

- This is not a law of physics, but an observation that has guided the industry for decades.
- **Current problem:** We are approaching the "Atomic Wall". If we make transistors smaller than 1-2 nm, electrons start to "jump" through the walls of the circuits due to quantum physics (a phenomenon called *Quantum Tunneling*), making the processor unpredictable.

💡 QualiAdept Analogy: "Drawing on the Grain of Rice"

- 💡 Imagine you want to write the entire history of the world on a single grain of rice.
- At first, you use a **normal pen** (70s technology). You can only write a few words.
 - Then, you use a **sewing needle** (90s). You're already writing a few pages.
 - Today, you use a **microscopic laser** who writes letters the size of molecules.

If you try to write even smaller, you'll reach the point where your letter is made up of a single atom. If you try to write smaller than that, the atom breaks or becomes unreadable. That's the limit humanity has reached with silicon.

👤 The Tester's Eye: "Binning" – Not all chips are created equal

In the factory, the process is so sensitive that a single grain of dust can destroy a processor. For this reason, on the same silicon disk (wafer), some chips come out perfect, others with small defects.

- **The Binning Process:** Manufacturers test each chip after manufacturing.
 - Those that reach high frequencies without heating up become **Core i9** or **Ryzen 9**.
 - Those that have a defective core or are not stable at high speeds are "limited" from the factory and sold as **Core i5** or **i3**.
- **Moral for Testers:** Sometimes a performance "bug" is not from the software, but from the physical quality of the silicon (the so-called "Silicon Lottery").

🔍 Reflection exercise

Think about a laptop from 10 years ago. It was thick, heavy, and had a 2-hour battery life. Today's laptop is thin, powerful, and has a 15-hour battery life.

- *What is the role of "nanometers" in this transformation? What would have happened if we had stayed with the manufacturing process of 2010?*

